

디지털 신호 처리 (EEE4420.01-00) – Simulation5 과제

※ 주어진 음성 신호  $x[n]$  ('input.wav',  $F_s = 8,000\text{Hz}$ ), 필터  $h[n]$  ('h.txt')와 30dB SNR을 가지는 Gaussian noise  $u[n]$  ('noise.wav')를 이용하여 진행하는 실험입니다. 신호  $y[n]$ 는 음성 신호  $x[n]$ 이 필터  $h[n]$ 을 통과한 후 노이즈  $u[n]$ 에 간섭을 받아 열화된 신호입니다. 이는 다음의 식과 같이 표현될 수 있습니다.

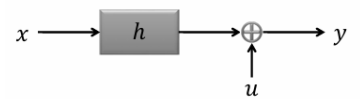
$$y[n] = h[n] * x[n] + u[n] \quad \cdots \text{eq. (1)}$$

[Hint]

- wav 파일을 읽어 들일 때는 `x = audioread('x.wav');` 함수를 사용하세요.
- txt 파일을 읽어 들일 때는 `h = readmatrix('h.txt');` 함수를 사용하세요.
- PSD(=Power spectrum)는  $E\{|X(w)|^2\}$ 와 같이 구할 수도 있지만, 이번 시뮬레이션에서는 Spectrum  $X(w)$  **magnitude** 제공  $|X(w)|^2$ 으로 근사하여 사용하세요.
- log scale에서 그래프를 그릴 때,  **$\log(1+\text{PSD})$**  식을 사용하세요.
- 신호를 필터링할 때, Convolution property를 이용하여 **주파수 영역에서 필터링** 하세요.

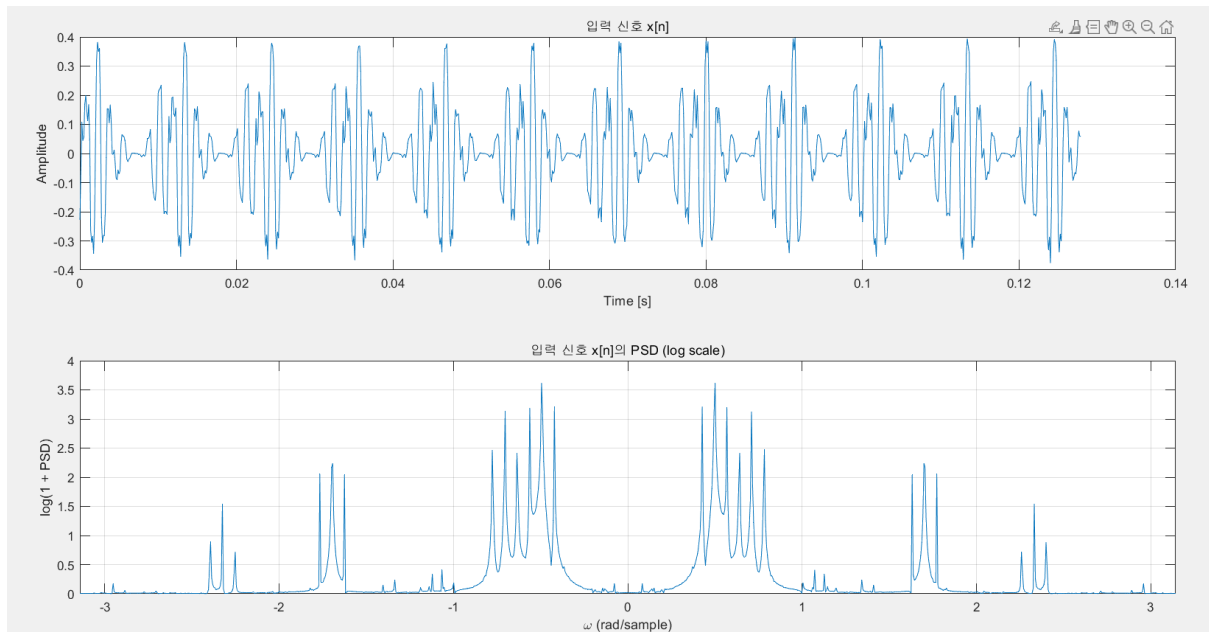
Wiener Least Squares Filter란 오른쪽 그림과 같은 system이 존재할 때, system을 통과하고 noise가 섞인 신호  $y$ 로부터,  $x$ 를 복원하는 Filter이다. 또한, Winer Filter의 수식은  $S_{xx}$ (원 신호  $x$ 의 PSD)를 이용하여 구성되는 것으로 Winer Filter는 Ideal한 상황을 가정하는 Filter이다. 즉, 현실에서  $x$ 를 찾기 위해서 쓰는 Filter인데, Filter의 수식에서  $x$ 의 PSD를 요구하기에, 실제로는 불가능한 Filter라는 것이다.

$$y[n] = h[n] * x[n] + u[n]$$



[이번 실습에서 log scale은  $\log_{10}(1+\text{PSD})$ 를 사용하였습니다. Discussion에  $\log(1+\text{PSD})$ 를 사용한 plot 또한 남겨두었습니다.]

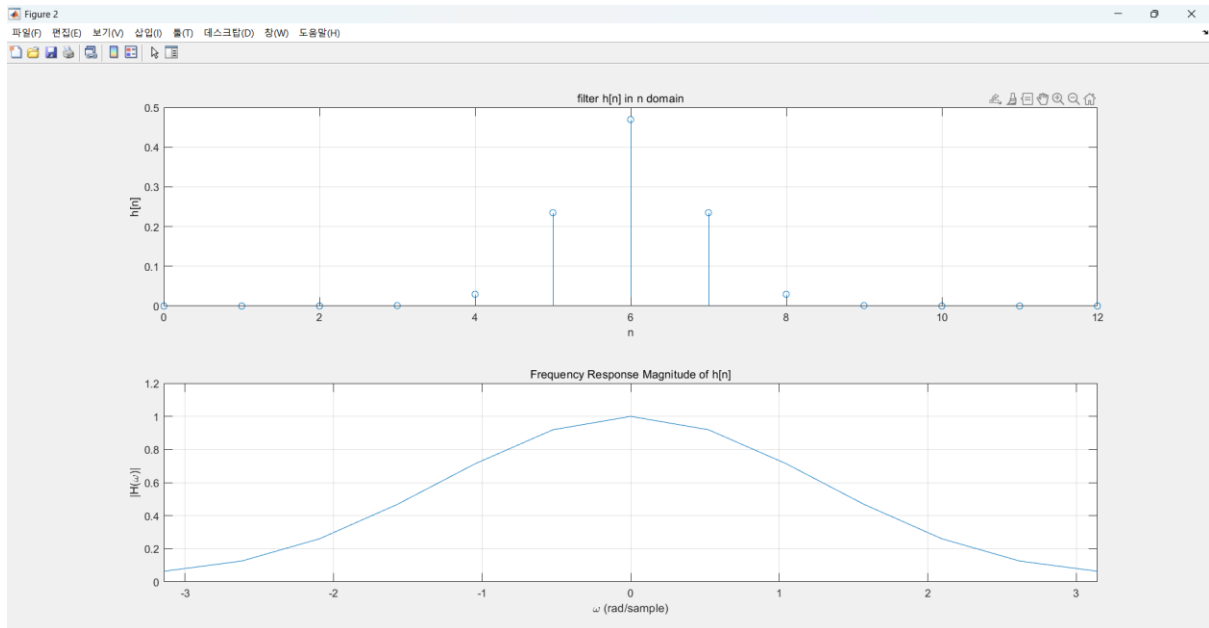
1. 주어진 입력 신호  $x[n]$ 의 파형을 그린 후 PSD(Power Spectral Density) 그래프를 log scale로 그리세요.



위 plot은  $x[n]$ 을 time domain에서 그린 파형이다. 주어진 입력 신호  $x[n]$ 은 time domain에서는 주기적인 모습을 보인다.

아래 plot은  $x[n]$ 을 FFT하여 생성한  $X(\omega)$ 의 제곱을 통해 구한 PSD를 plot한 그림이다. 아래 plot의 경우에는 평범한 신호와는 다르게 0 rad/sample에 Magnitude가 0에 가깝고,  $\frac{\pi}{5}, \frac{\pi}{2}, \frac{2\pi}{3}$  즈음에 큰 Magnitude를 가지는 것을 볼 수 있다.

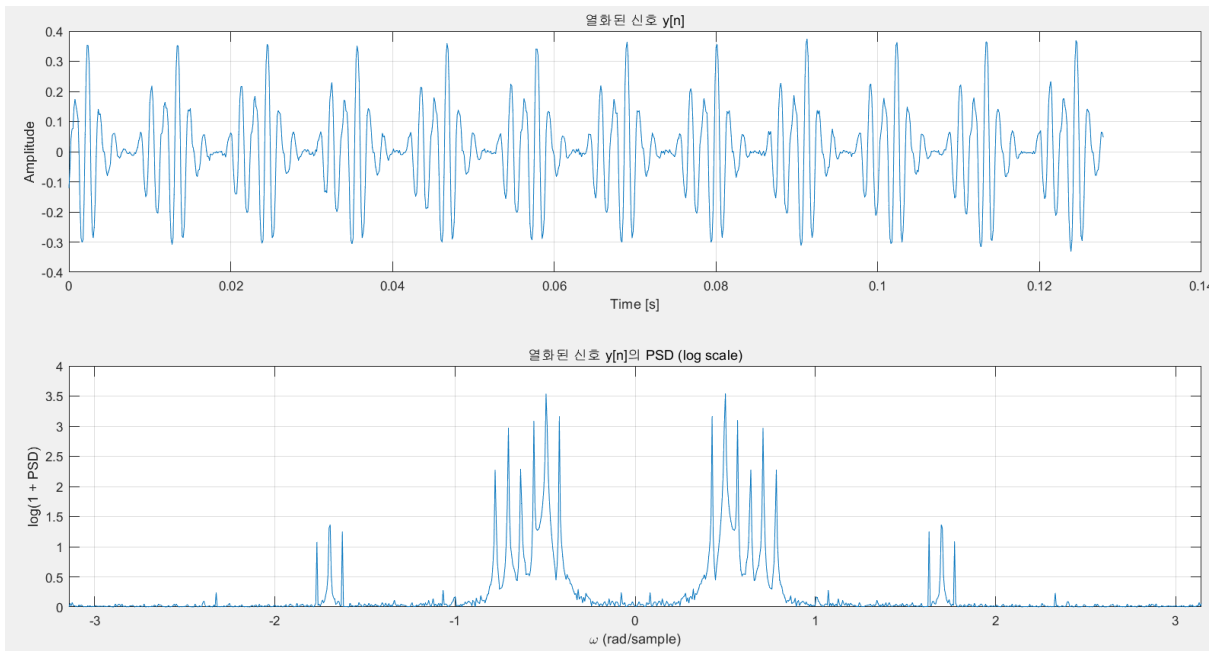
2. 필터  $h[n]$ 을 그리고 Frequency Response  $H(w)$ 의 magnitude 그래프를 linear scale로 그리세요.



위 plot은  $n$  domain에서  $h[n]$ 의 모습이다. 현재 MatLab에서 shift를 하지 않아,  $n = 6$ 에 가장 큰 magnitude를 가지는 sample이 존재하고,  $n = 6$ 기준으로 대칭적으로 점차 magnitude가 감소하는 형태를 보인다. 이는 sinc함수를 windowing한 모습이란 비슷하므로, 이를 통해  $h[n]$ 을 FFT시 LPF와 같은 모습을 보일 것을 예상할 수 있다.

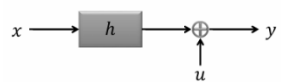
아래 plot은 frequency domain에서  $F\{h[n]\} = H(w)$ 의 모습을 보여준다. Frequency domain에서 저주파 대역에서 큰 Magnitude를 가지고, 고주파 대역에서 작은 Magnitude를 가지기에, 앞서 예상했듯이,  $h[n]$ 은 LPF와 같이 동작함을 알 수 있다.

3. 신호  $x[n]$ , 필터  $h[n]$ , 노이즈  $u[n]$ 을 이용하여 열화된 신호  $y[n]$ 을 만드세요.  $y[n]$ 의 파형을 그린 후,  $y[n]$ 의 PSD 그래프를 log scale로 그리세요.

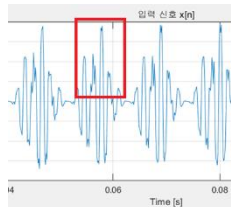


$y[n]$ 의 경우 오른쪽 그림과 같이,  $y[n] = h[n] * x[n] + u[n]$ 을 거쳐서 생성되는 신호이다. 여기서  $h[n]$ 은 system,  $u[n]$ 은 Noise를 의미한다. 이번 실습에서는 Gaussian Noise를 사용한다.

$$y[n] = h[n] * x[n] + u[n]$$

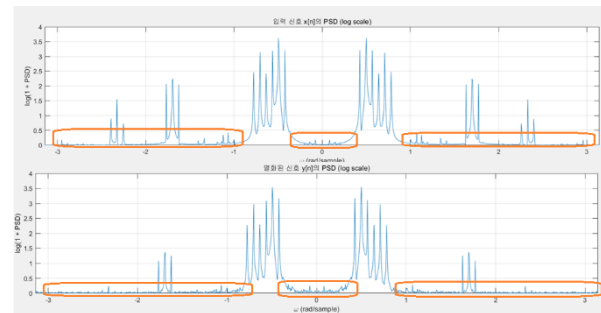


LPF로서 작동하는  $h[n]$ 을 거쳤기 때문에, time domain에서는 원 신호가 가지고 있던, 큰 amplitude에서의 고주파의 움직임이 사라져 더욱 부드럽게 변했다.

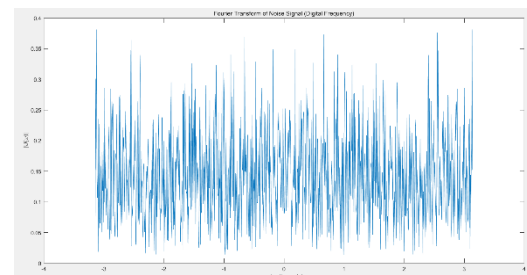


Frequency Domain을 분석하면, 고주파 대역에 있던 신호( $\frac{2\pi}{3}$  구간)는 magnitude가 거의 0에 가깝게 변하였고,  $\frac{\pi}{2}$  구간에 있던 신호 또한, magnitude가 많이 감소한 것을 확인할 수 있다.

더불어, Noise의 영향도 확인할 수 있다. Frequency Domain의 plot에서 오른쪽 그림의 주황색 박스 친 부분들을 살펴본다.  $x[n]$ 에서는 신호들이 많이 튀지 않지만,  $y[n]$ 에서는 자잘하게 신호들이 튀는 모습을 볼 수 있다.



이 page 가장 오른쪽 아래의 plot은 Noise의 Frequency Domain에서의 파형인데, magnitude 0.05 ~ 0.35사이에 분포해 있는 것을 확인할 수 있다. 이로써, 이 noise는 완벽하지는 않지만, flat한 frequency response를 보여주는 white noise의 성격을 지니고 있다고 생각할 수 있을 것 같다.



[Code 구현]

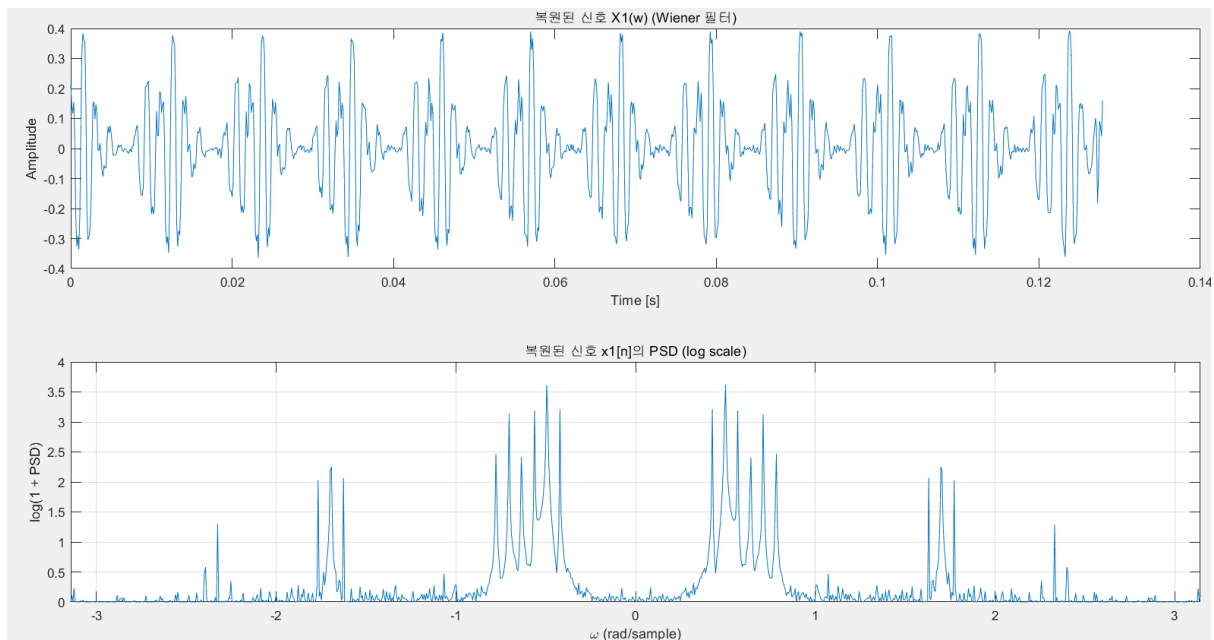
위 plot의 경우에는  $y[n] = h[n] * x[n] + u[n]$  수식에 따라, conv함수를 이용하여 진행하였다. 만약, convolution property에 따라,  $Y(w) = H(w) * X(w) + U(w)$  로  $y[n]$ 을 구하게 되면, plot이 약간 다르게 나오는데, 이는 Discussion에서 다루겠다.

4. Wiener filter를 이용하여 열화된 신호  $y[n]$ 으로부터  $\hat{x}_1[n]$ 을 복원합니다. 복원된 신호  $\hat{x}_1[n]$ 을 그리고  $\hat{x}_1[n]$ 의 PSD 그래프를 log scale로 그리세요. Wiener Filter를 구현할 때는 아래의 식을 참고하세요.

$$H_{wiener}(\omega) = \frac{H^*(\omega)}{|H(\omega)|^2 + \frac{S_{uu}(\omega)}{S_{xx}(\omega)}}, \quad S_{uu}(\omega) = 5 \times 10^{-3} \quad \dots eq.(2)$$

$$\hat{X}_1(\omega) = H_{wiener}(\omega) \cdot Y(\omega), \quad \hat{x}_1[n] = IDFT[\hat{X}_1(\omega)] \quad \dots eq.(3)$$

※ 이때  $x[n]$ 은  $y[n]$ 을 통해 복원하고자 하는 원본 신호이기 때문에 복원 과정에 활용될 수 없습니다. 따라서,  $S_{xx}(\omega)$  대신  $S_{yy}(\omega)$ 를 이용하여 Wiener filter를 설계하세요.



[왜  $S_{xx}(\omega)$ 를 사용하지 못하는 가?]

$S_{xx}(\omega) = \mathcal{F}\{\gamma_{xx}[m]\}$ 으로 원본 신호  $x[n]$ 의 auto correlation을 FT 취한 꼴이다. 즉,  $S_{xx}(\omega)$ 를 사용하려면, 원본신호의 형태를 알아야 한다. 물론, 강의에서의 말씀을 빌리면, Power Spectrum이기에 원본 신호 중 50%의 정보만 필요한 것이다. 하지만, 50%라고 해도 원본 신호의 정보가 필요하기에, 현재 원본 신호의 정보를 전혀 모르는 상태로는  $S_{xx}(\omega)$ 를 사용할 수 없다.

따라서,  $S_{xx}(\omega)$ 의 정보를 가장 많이 포함하고 있는  $S_{yy}(\omega)$ 를 대신 식에 대입하여 사용한다.

[Winer Filter의 역할 및 일반적인 Least Square Inverse Filter와의 차이점]

Winer Filter는 기본적으로 Inverse Filter이다. 즉,  $h[n]$ 의 inverse인  $h_i[n]$  system을 구성한다. 하지만, Least Square Inverse와는 매우 유사하지만, 결정적인 차이가 있다. Winer Filter는  $S_{xx}(\omega)$  와  $S_{uu}(\omega)$

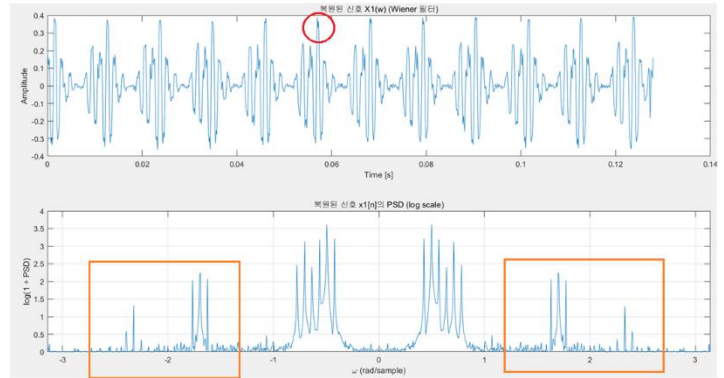
에 의해 자동적으로 계산되어 결정되는 Regularization term이 존재한다는 것이다. 이 Regularization term은 Noise의 영향으로 의도치 않게 Spectrum의 Magnitude가 0이 되는 것을 방지하는데, Noise의 영향이 어느 정도인지에 따라 Regularization term의 크기가 달라진다. 보통의 Least Square Inverse Filter는 수식적으로 Regularization term이 산출되지 않아, 이 term을 정하기 애매하지만, winer filter는 Regularization term이  $\frac{S_{uu}(\omega)}{S_{xx}(\omega)}$ 로 수식적으로 나온다.

[Plot 분석]

열화된 신호  $y[n]$ 에서는 LPF 및 Noise에 의해 신호가 부드러워지고, 자잘하게 신호들이 튀는 모습이 존재하였다.

Winer Filter로 복원한 신호  $\hat{x}_1[n]$ 에서는 amplitude가 클 때 존재하였던 고주파의 파형이 다시 복원된 것을 볼 수 있다(빨간색 원).

Frequency Domain을 보게 되면, LPF에 의해 크기가 작아졌던,  $\frac{2\pi}{3}, \frac{\pi}{2}$  구간의 신호들도 원래 크기보다는 아주 살짝 작지만, 거의 유사한 정도까지 Magnitude가 복원된 모습을 볼 수 있다(주황색 박스).



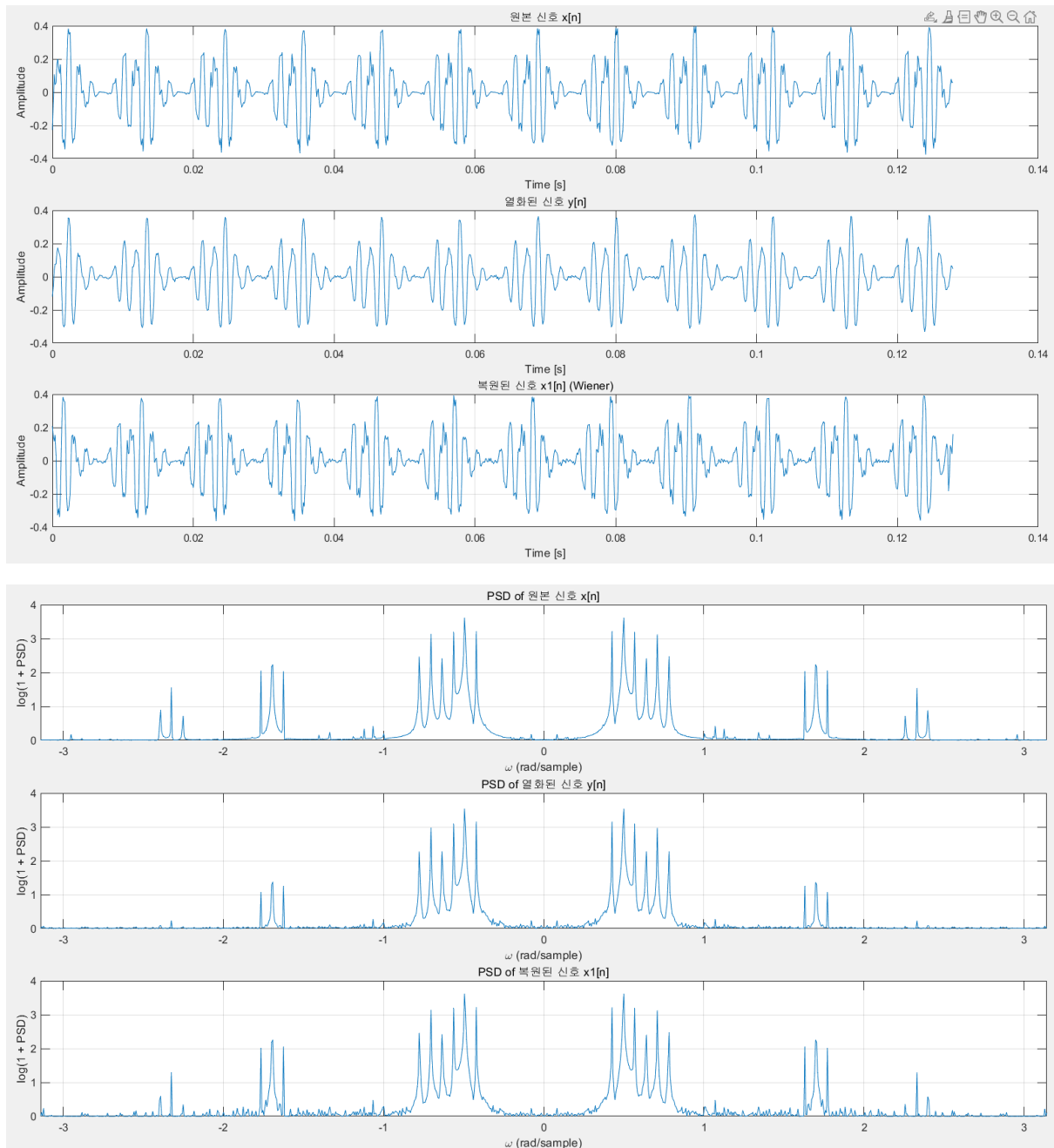
하지만, Noise의 영향이 얼마나 줄었는지는 육안으로 힘들다고 판단되었다. 왜냐하면,  $x[n]$ 의 frequency response에서 보이지 않았던 자잘하게 신호들이 튀는 모습이 여전히 Winer Filter를 거친  $\hat{x}_1[n]$ 에서도 보였기 때문이다. 하지만,  $y[n]$ 에서와 경향성이 다르게 분포되어 있으므로, 어느정도 Noise에 영향을 주었음은 살펴볼 수 있다.

이로써, 비록  $S_{xx}(\omega)$ 를 사용하지는 않았지만,  $S_{yy}(\omega)$ 를 통해서도 잘 복원된 모습을 볼 수 있다.

[왜  $S_{yy}(\omega)$ 를 통해서도 잘 복원되었는가?]

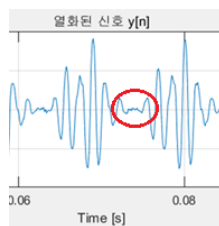
현재 system인  $h[n]$ 은 LPF의 형태를 띈다. 그리고  $x[n]$ 은 주파수 0에는 정보가 거의 없었지만, 0 주위의 저주파 대역에 많은 정보들이 존재하였다. 즉,  $x[n]$ 이 LPF를 거치면 많은 정보들이 살아남기에,  $S_{yy}(\omega)$  또한  $x[n]$ 의 정보를 많이 포함한 것이다.

5. 원본 신호  $x[n]$ , 열화된 신호  $y[n]$ , 복원된 신호  $\hat{x}_1[n]$ 의 파형과 PSD 그래프를 비교하고 분석하세요.



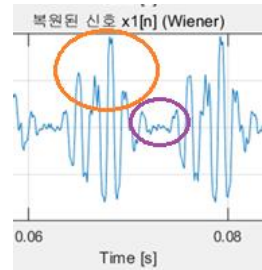
[ $x[n]$ 과  $y[n]$ 과  $\hat{x}_1[n]$  파형 비교]

$y[n]$ 의 경우에는  $x[n]$ 이 LPF를 거치고 Noise가 섞였다. 따라서,  $x[n]$ 의 amplitude가 클 때 존재 하였던, 고주파 성분들이 없어서 부드러워진 것을 볼 수 있다. 더불어, Noise의 영향으로, 오른쪽 그림의 빨간색 원에서 보인 것과 같이 amplitude가 0에 가까운 부분에 작은 성분들의 신호가 생긴 것을 볼 수 있다. 더불어, Amplitude가  $x[n]$ 에 비해 전체적으로 약간 줄었지만, LPF를 거쳐 많은 정보들이 살아남았기에 크게 줄지는 않은 모습을 볼 수 있다.



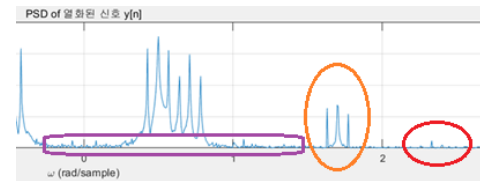


$\hat{x}_1[n]$ 의 경우에는  $y[n]$ 을 Winer Filter를 거쳐서 복원된 신호이다. Inverse Filter로 작용하는 Winer Filter를 거쳤기 때문에, 고주파 성분들이 복원되어, 주황색 원에 보이는 것을 확인할 수 있다. 하지만,  $\frac{S_{uu}(\omega)}{S_{yy}(\omega)}$ 를 통하여 Noise의 영향이 최소화되어야 하지만, 현재 파형을 통해서는 그 효과를 짐작하기 어렵다(보라색 원).

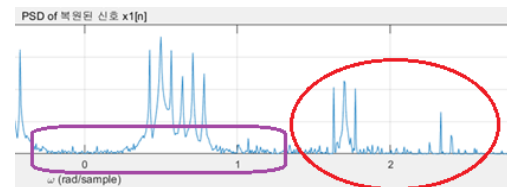


[ $x[n]$ 과  $y[n]$ 과  $\hat{x}_1[n]$  PSD 비교]

$y[n]$ 의 PSD를  $x[n]$ 의 PSD와 비교해본다. 먼저,  $h[n]$ 의 영향에 의해 저주파 대역에 있는 Spectrum의 Magnitude는 많이 줄지 않았지만, 고주파 대역에 있는 Spectrum의 Magnitude는 많이 줄은 것을 확인할 수 있다 (주황색 원, 빨간색 원). 특히, 빨간색 원의 대역에 존재했던 신호는 거의 없어졌다고 볼 수 있다. Noise의 영향은 보라색 박스에서 확인할 수 있다. Noise가 추가되어, 매우 작은 크기의 Spectrum들이 일정하게 전 주파수 대역에 퍼져 있는 것을 확인할 수 있다.

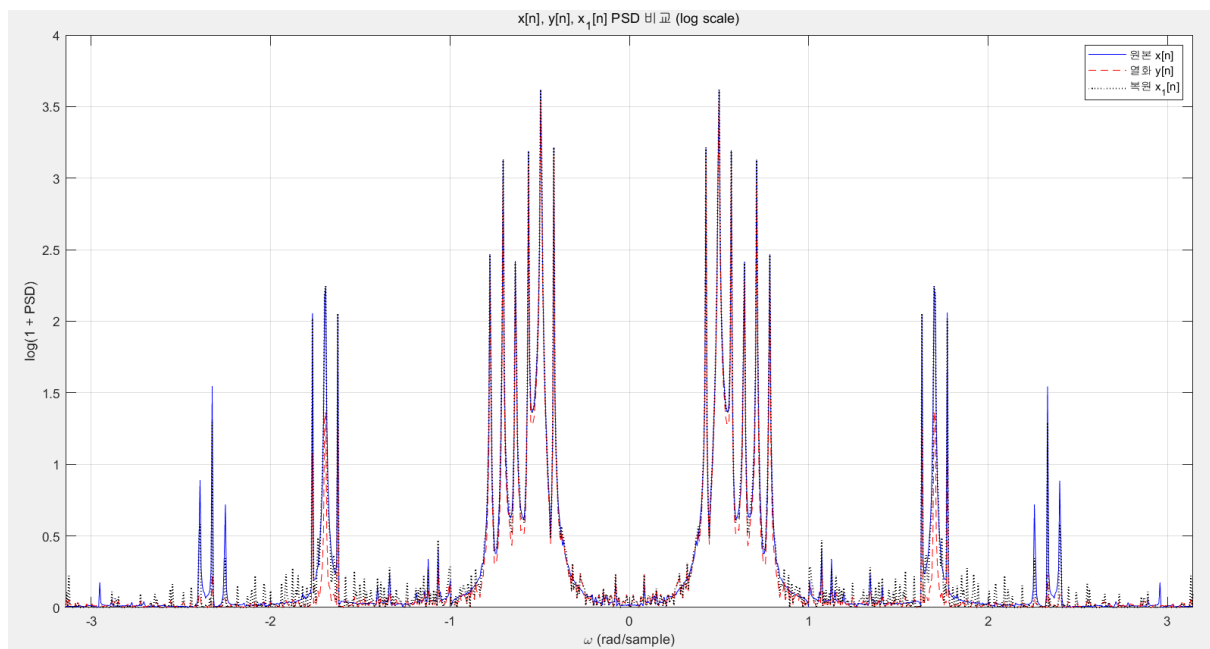
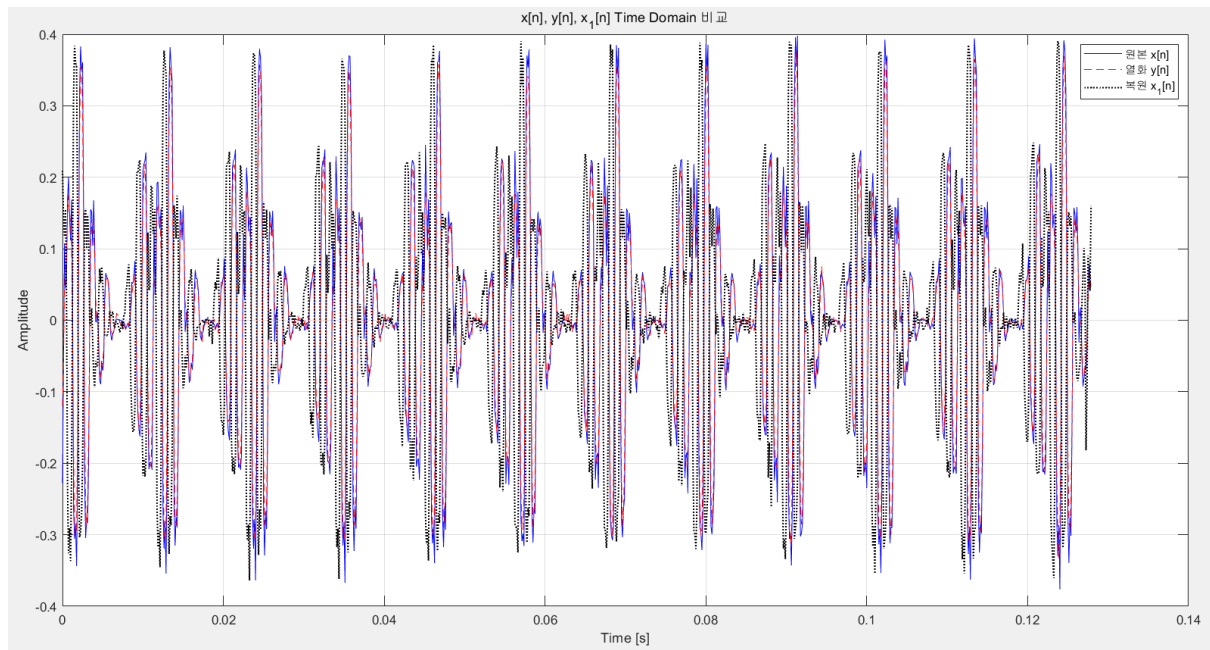


$\hat{x}_1[n]$ 의 PSD를 살펴본다. Inverse Filter로 작용하는 Winer Filter를 거쳤기 때문에, 고주파 성분들이 복원되어 빨간색 원에 보이는 것과 같이 고주파 성분들이 다시 복원된 것을 확인할 수 있다. 하지만, Magnitude의 경우에는 원 신호보다 조금 작은 것을 확인할 수 있다.



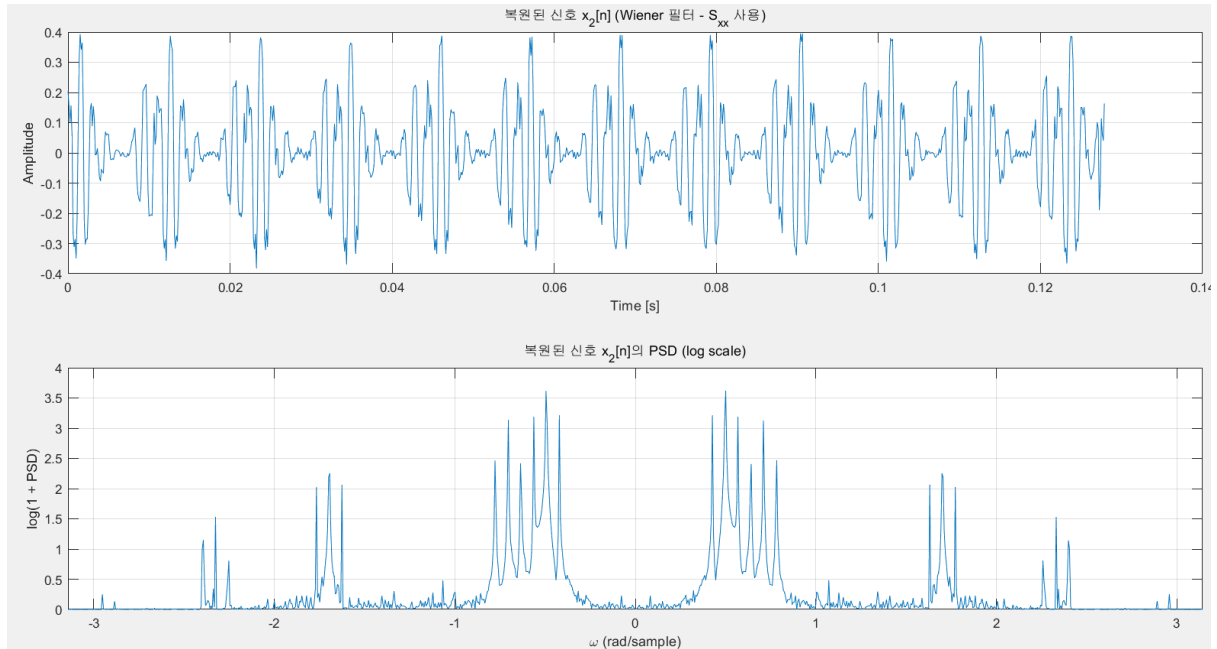
이는  $S_{yy}(\omega)$ 를 통하여 복원되었기 때문에,  $x[n]$  원 신호의 정보의 손실로 인한 결과라고 예상된다. 더불어, 보라색 박스를 보면, 아직도, 작은 magnitude의 spectrum들이 전 주파수 대역에 퍼져 있는 것을 확인할 수 있다. 따라서, PSD 파형으로는 Winer Filter가 얼마나 Noise를 억제했는지는 파악하기 힘들다.

[참고: 겹쳐서 그린 그래프]



겹쳐서 그린 time domain을 보게 되면, 복원한 신호가 약간의 delay를 가지는 것을 볼 수 있다. 이는,  $\frac{S_{uu}(\omega)}{S_{yy}(\omega)}$ 를 통해 x[n]의 Power에 대한 정보만 Winer Filter에서 사용하였고, 나머지 절반의 정보인 Phase 정보는 사용하지 못했기 때문이라고 예상한다.

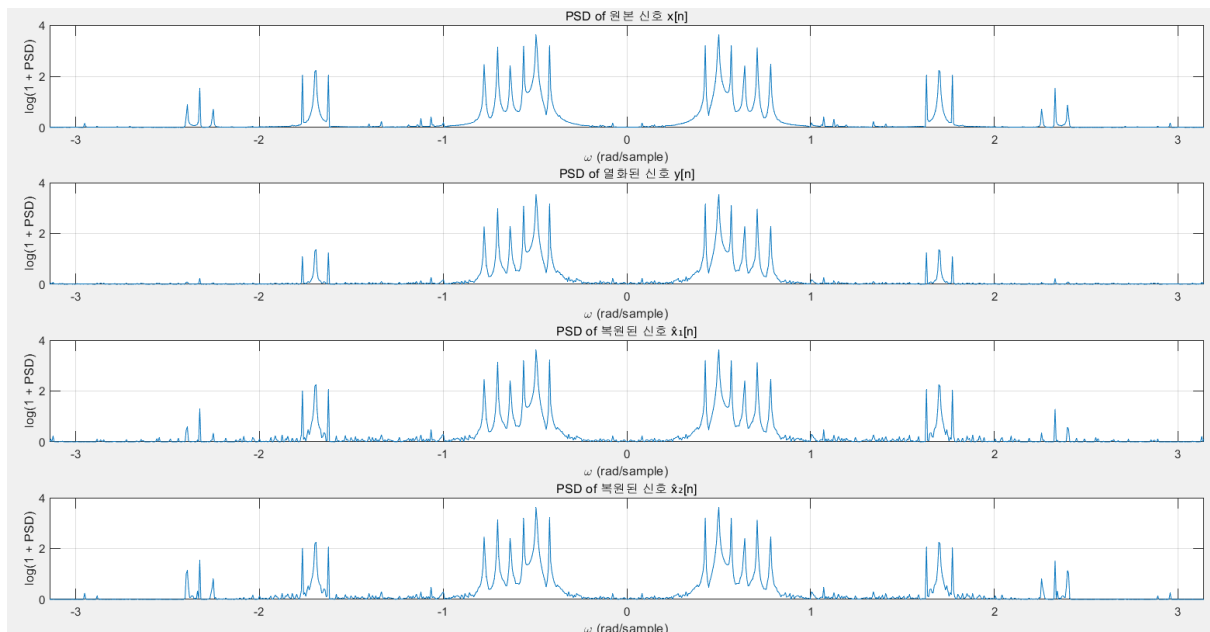
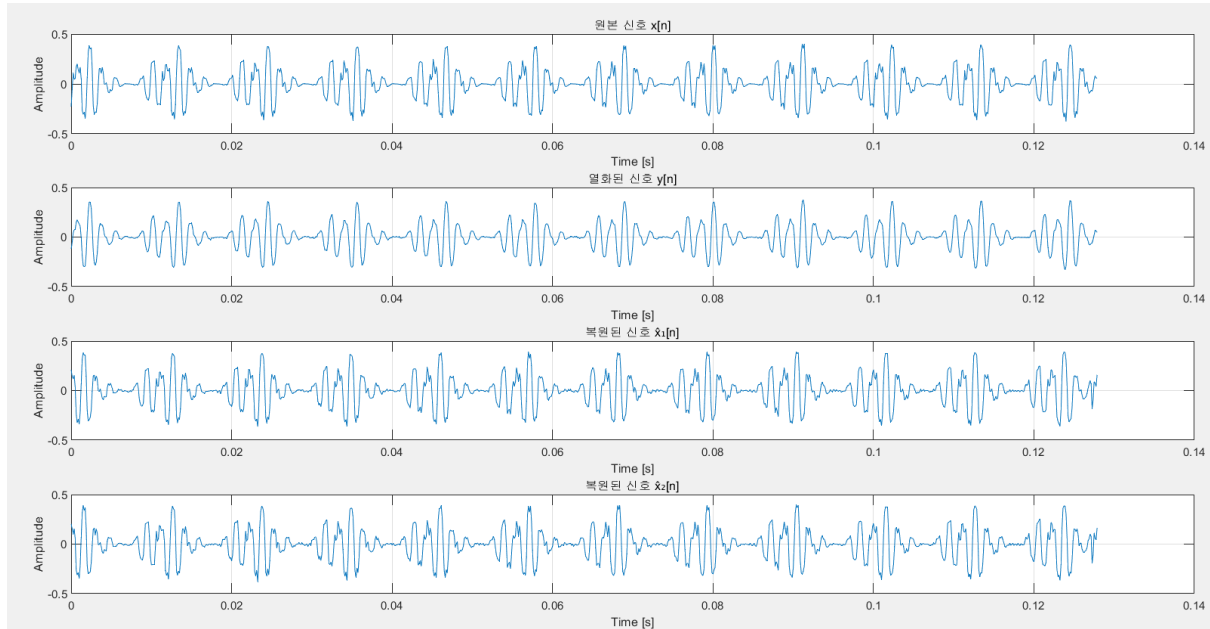
6. 이번엔  $S_{yy}(\omega)$ 를 이용한 문제 4와 다르게,  $S_{xx}(\omega)$ 를 직접 활용하여 Wiener filter를 설계하여 열화된  $y[n]$ 으로부터  $\hat{x}_2[n]$ 을 복원합니다. 복원된 신호  $\hat{x}_2[n]$ 을 그리고  $\hat{x}_2[n]$ 의 PSD 그래프를 log scale로 그리세요.



$\frac{S_{uu}(\omega)}{S_{yy}(\omega)}$ 가 아닌  $\frac{S_{uu}(\omega)}{S_{xx}(\omega)}$ 를 사용하였기 때문에, 원 신호  $x[n]$ 의 Power에 관한 정보는 온전히 사용하게 된다. 따라서,  $\frac{S_{uu}(\omega)}{S_{xx}(\omega)}$ 를 이용한 Winer Filter는 Ideal한 Filtering 효과를 기대할 수 있고,  $\hat{x}_1[n]$ 과 비교 시  $\hat{x}_2[n]$ 은 더욱 잘  $x[n]$ 을 복원했을 것이라고 예상할 수 있다.

결과 비교 및 분석은 문제 7번에서 진행한다.

7. 원본 신호  $x[n]$ , 열화된 신호  $y[n]$ , 복원된 신호  $\hat{x}_2[n]$ 의 파형과 PSD 그래프를 비교하고 분석하세요. 문제 4의 결과  $\hat{x}_1[n]$ 과는 어떤 차이점이 존재하는지도 서술하세요.



[ $x[n]$ 과  $y[n]$ 과  $\hat{x}_2[n]$  파형 비교]

$h[n]$ 에 의해 없어진, 고주파 성분 및 Noise 추가로 인한 변화를 중심으로 살펴본다. 더불어,  $x[n]$ 과  $y[n]$ 의 비교는 이전의 문제에서 했기에, 생략한다.



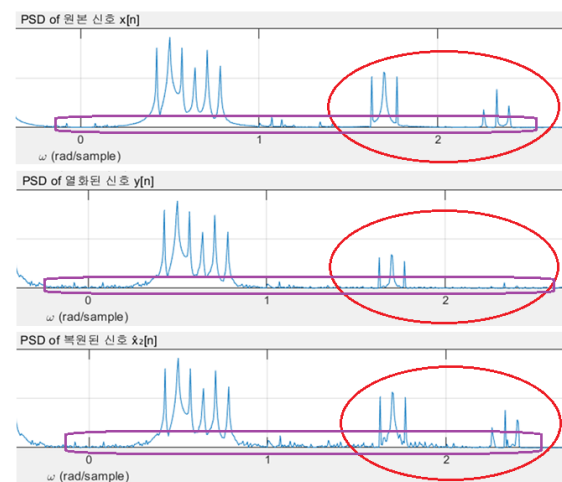
먼저,  $y[n]$ 과  $\hat{x}_2[n]$ 을 비교 시, LPF로 인하여 사라졌던 고주파 성분들이 다시 복원된 것을 볼 수 있

다. 하지만, Noise로 인해 생겼던, 작은 진동들은 Winer Filter를 통과한 파형에서도 다른 양상으로 나타났다. Winer Filter를 통과한 파형에서의 작은 진동들이 여전히 Noise로 작용할 지, 유의미한 신호로 나타날지는 파형을 통해서 분석하기는 어렵다.

다음으로,  $x[n]$ 과  $\hat{x}_2[n]$ 을 비교 시 두 파형은 매우 비슷한 모습을 보인다. 이는 즉, Winer Filter를 통하여 잘 복원되었다는 뜻이다. 더불어, Amplitude의 경우에도 거의 동일한 Amplitude를 가진다. 따라서, Power의 측면에서도 잘 복원되었다고 볼 수 있다. 하지만,  $\hat{x}_2[n]$ 에서는 Winer Filter를 통과한 파형에서의 작은 진동들이  $x[n]$ 과 다르게 존재하기에, 이 진동들이 Noise로 작용할 지는 직접 음성을 들어서 판단을 해야 할 것 같다.

#### [ $x[n]$ 과 $y[n]$ 과 $\hat{x}_2[n]$ PSD 비교]

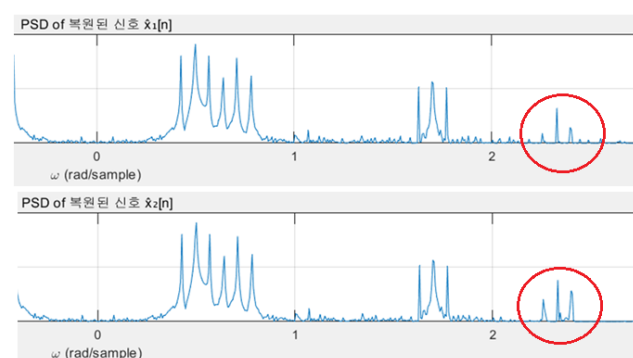
$y[n]$ 과  $\hat{x}_2[n]$ 의 PSD를 비교 시, 사실 상 앞서, time domain에서 분석했던 것과 같은 모습을 측정할 수 있다. 먼저, LPF로 인하여 사라졌던 고주파 성분들이 다시 복원된 것을 볼 수 있다(빨간색 원). 더불어, Noise로 인해 생겼던, 작은 진동들은 Winer Filter를 통과한 이후에도 남아있지만 magnitude가 다른 양상으로 나타났다.



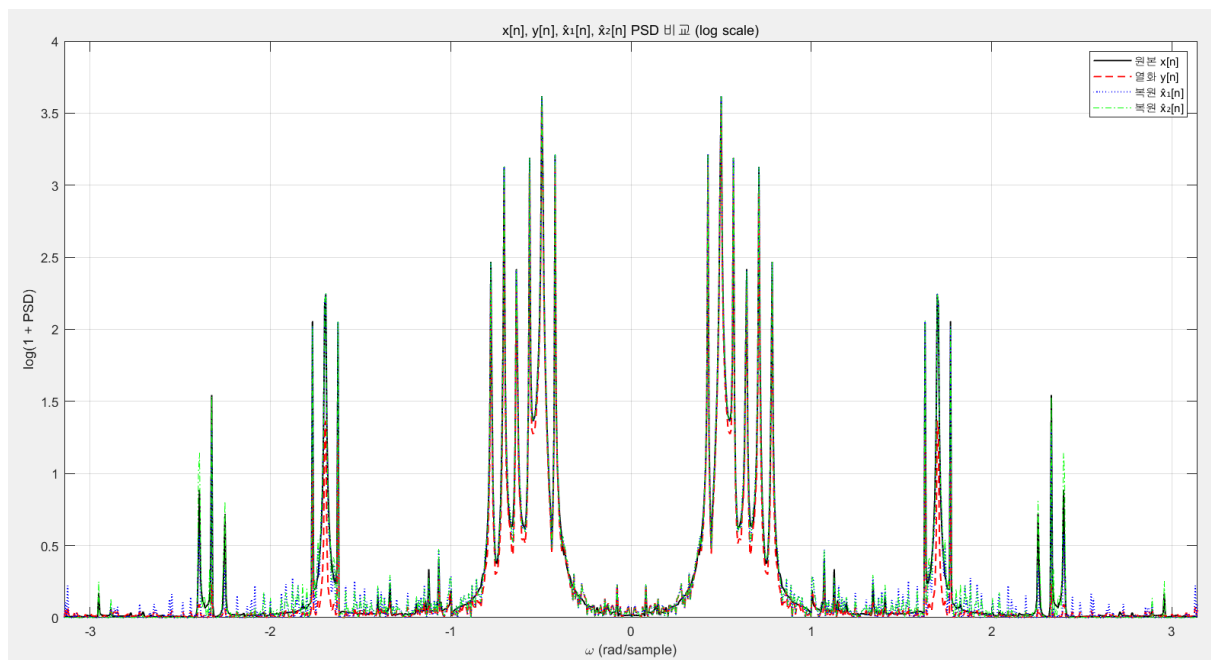
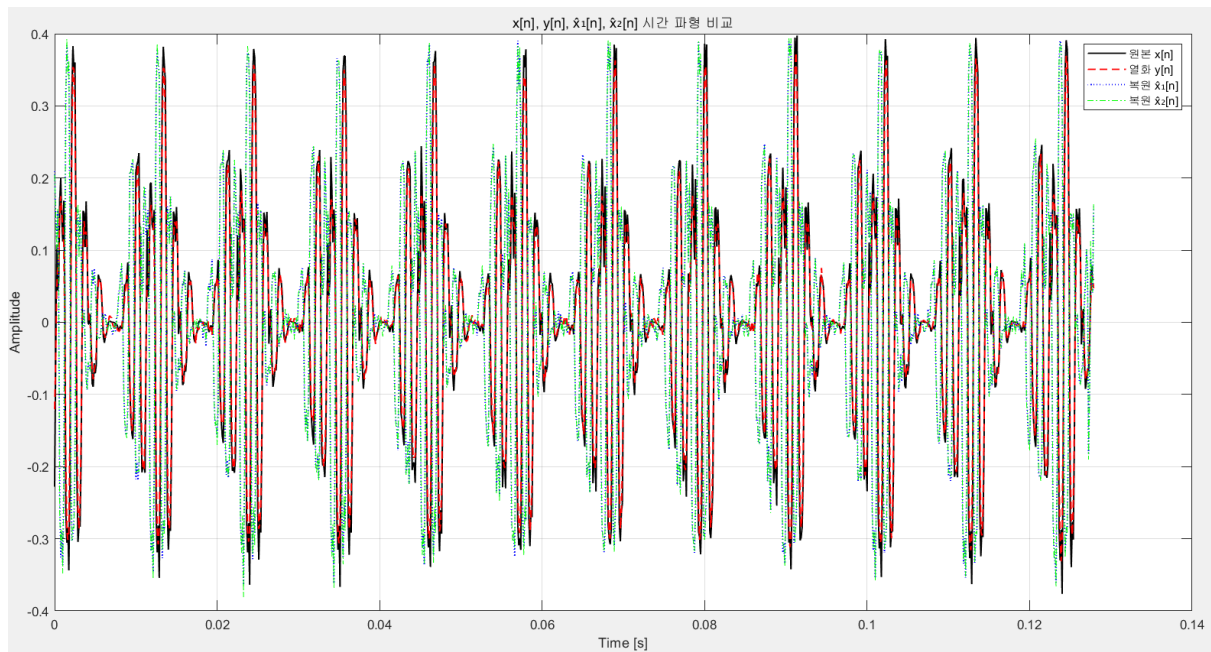
다음으로,  $x[n]$ 과  $\hat{x}_2[n]$ 을 비교 시 두 plot은 magnitude가 큰 spectrum끼리는 매우 비슷한 모습을 보인다. 이는 즉, Winer Filter를 Power의 측면 및 Frequency 측면에서 잘 복원되었다고 볼 수 있다. 하지만,  $\hat{x}_2[n]$ 에서는 Winer Filter를 통과한 파형에서의 작은 진동들이  $x[n]$ 과 다르게 존재한다.

#### [ $\hat{x}_1[n]$ 과 $\hat{x}_2[n]$ 파형 및 PSD 비교]

$\hat{x}_1[n]$ 과  $\hat{x}_2[n]$ 의 차이점은 신호를 복원 시  $\frac{S_{uu}(\omega)}{S_{yy}(\omega)}$ 을 썼는지  $\frac{S_{uu}(\omega)}{S_{xx}(\omega)}$ 을 썼는 지의 차이이다. 결국, 원 신호  $x[n]$ 의 Power 정보를 얼마나 가지고 있는지의 차이이다.  $S_{yy}(\omega)$ 의 경우  $X(\omega)$ 가  $H(\omega)$ 라는 LPF를 한번 거쳐서 나온 신호이기에, 고주파 대역의 Power가 매우 감소되었을 것이다. 그리고,  $S_{xx}(\omega)$ 의 경우에는 원신호의 PSD이기에 Power의 손실은 없었을 것이다. 이 차이점이 plot에서 바로 나타난다. 빨간색 원으로 보면,  $\hat{x}_1[n]$ 의 고주파 대역에서 복원된 신호의 Magnitude가  $\hat{x}_2[n]$ 의 고주파 대역에서 복원된 신호의 Magnitude보다 작은 것을 알 수 있다. Magnitude 측면을 제외한 나머지 부분에서는 거의 유사한 모습을 보인다.



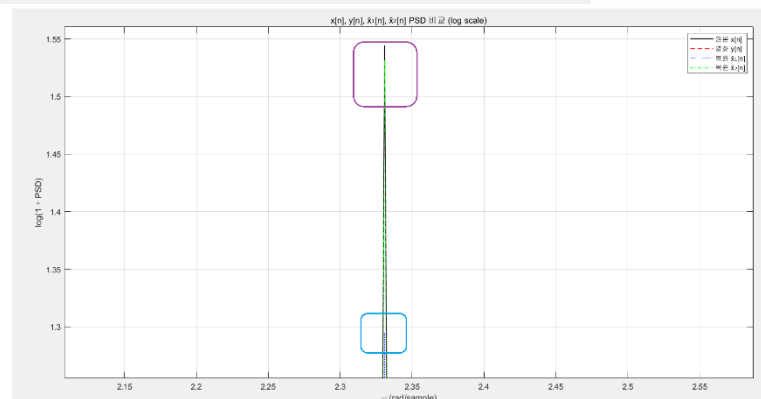
[참고: 겹쳐서 그린 그래프]



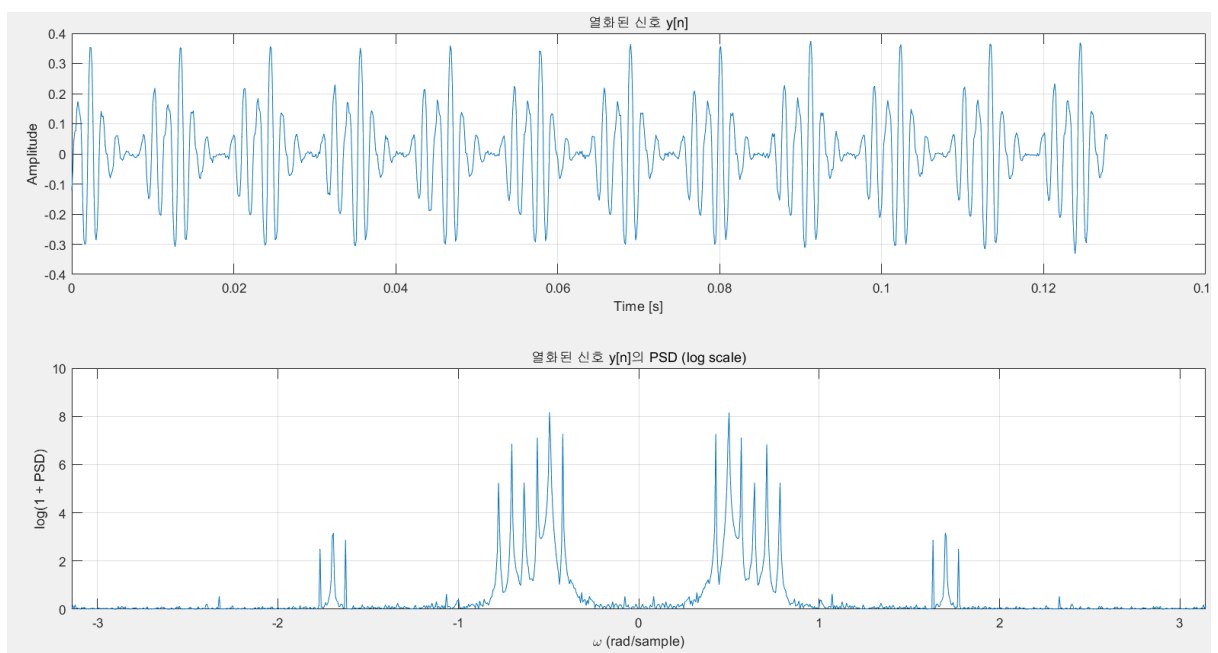
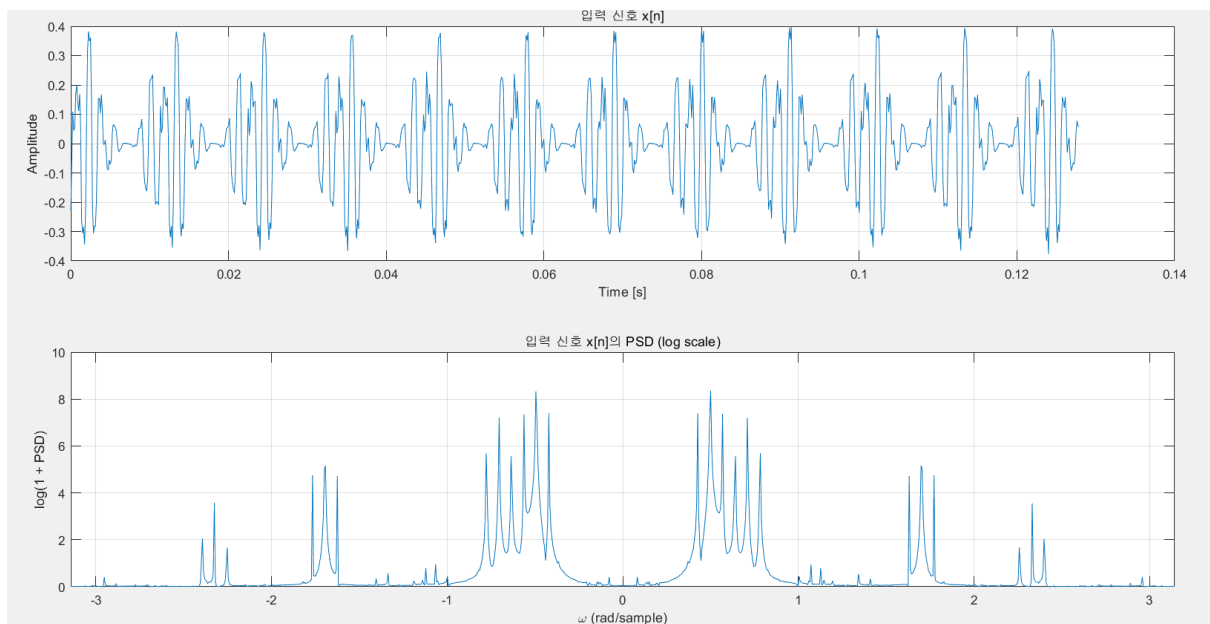
오른쪽 그림

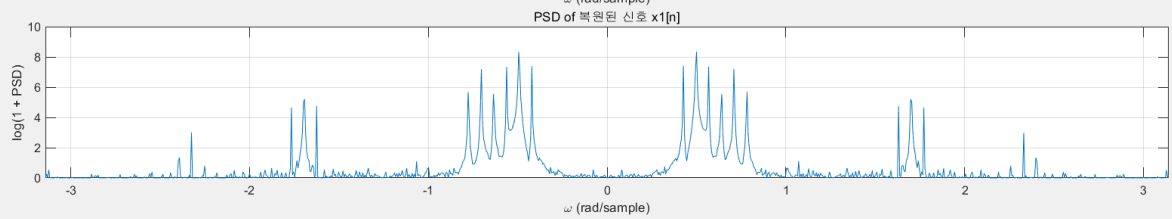
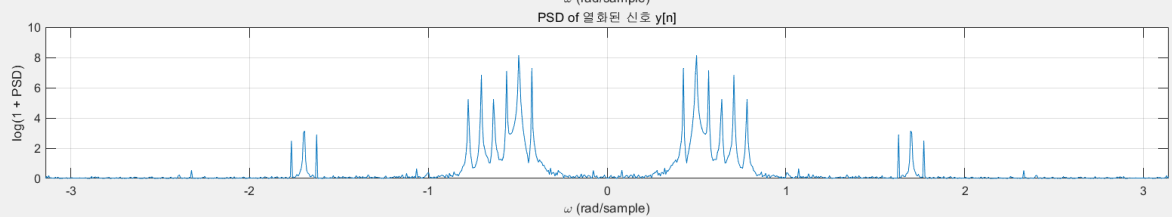
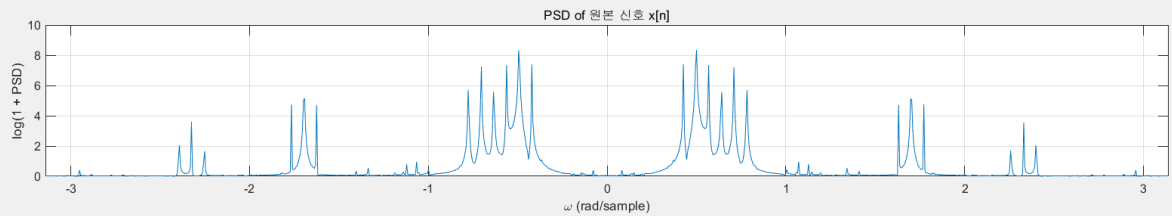
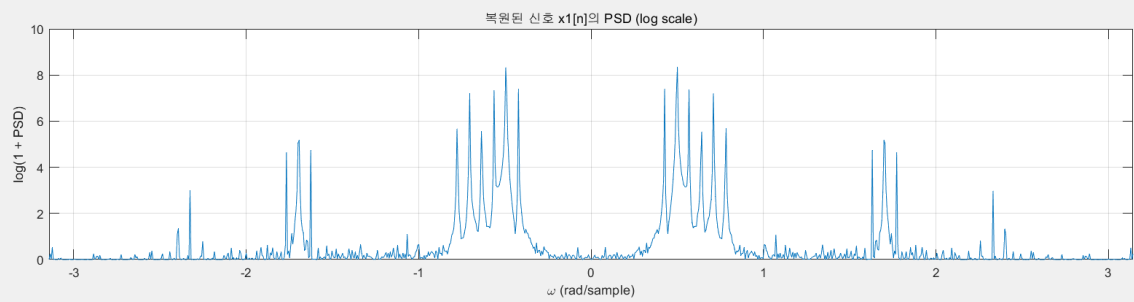
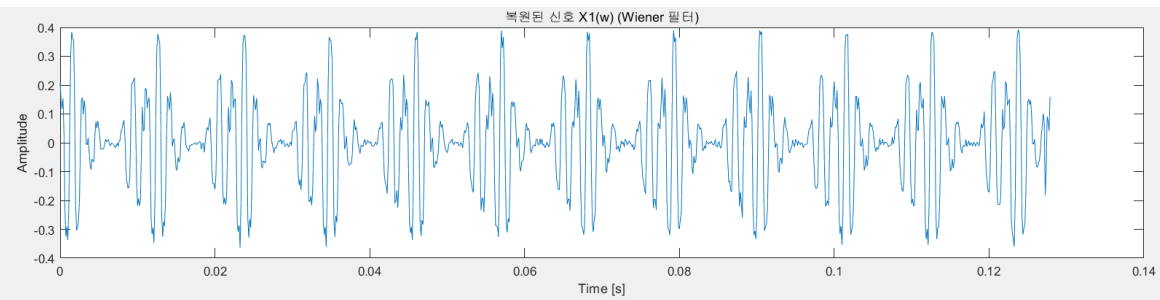
보라색 박스: 고주파 대역에서  $\hat{x}_2[n]$ 의 PSD 크기

파란색 박스: 고주파 대역에서  $\hat{x}_1[n]$ 의 PSD 크기

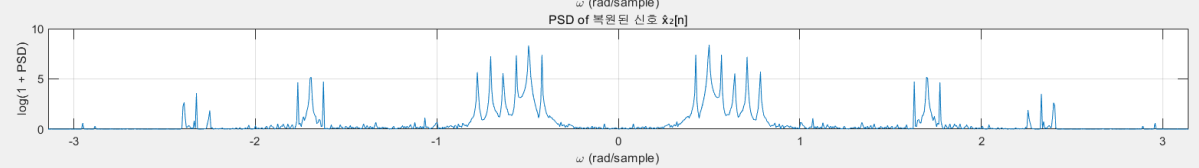
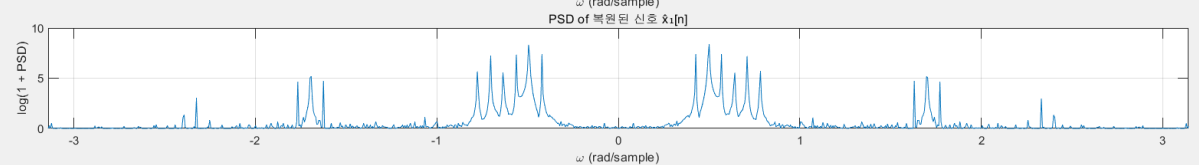
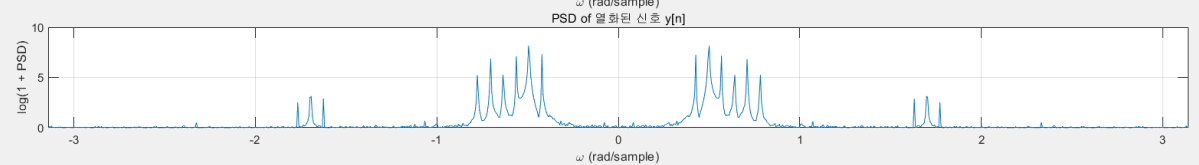
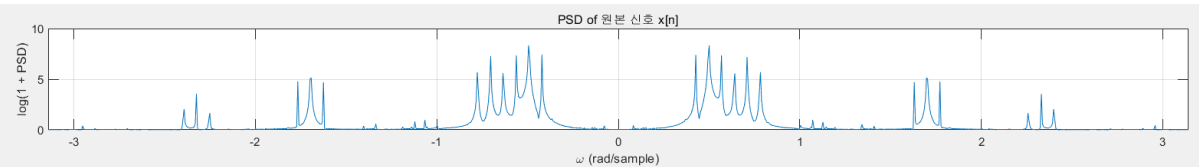
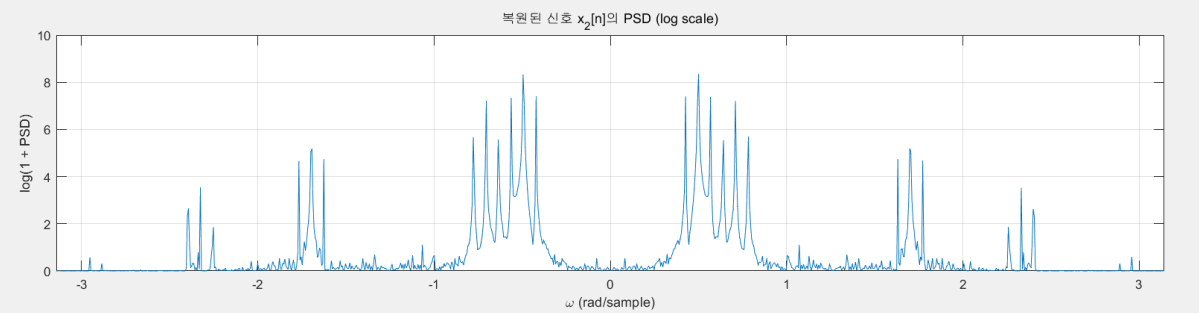
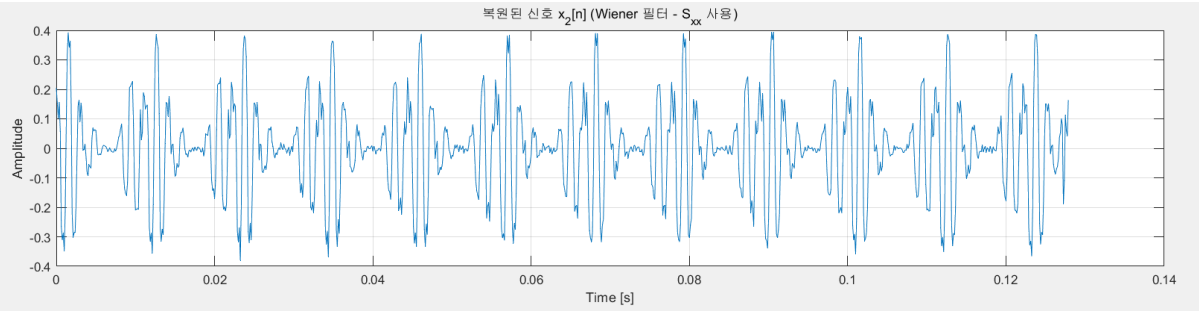


## [log(1+PSD)로 진행 시]

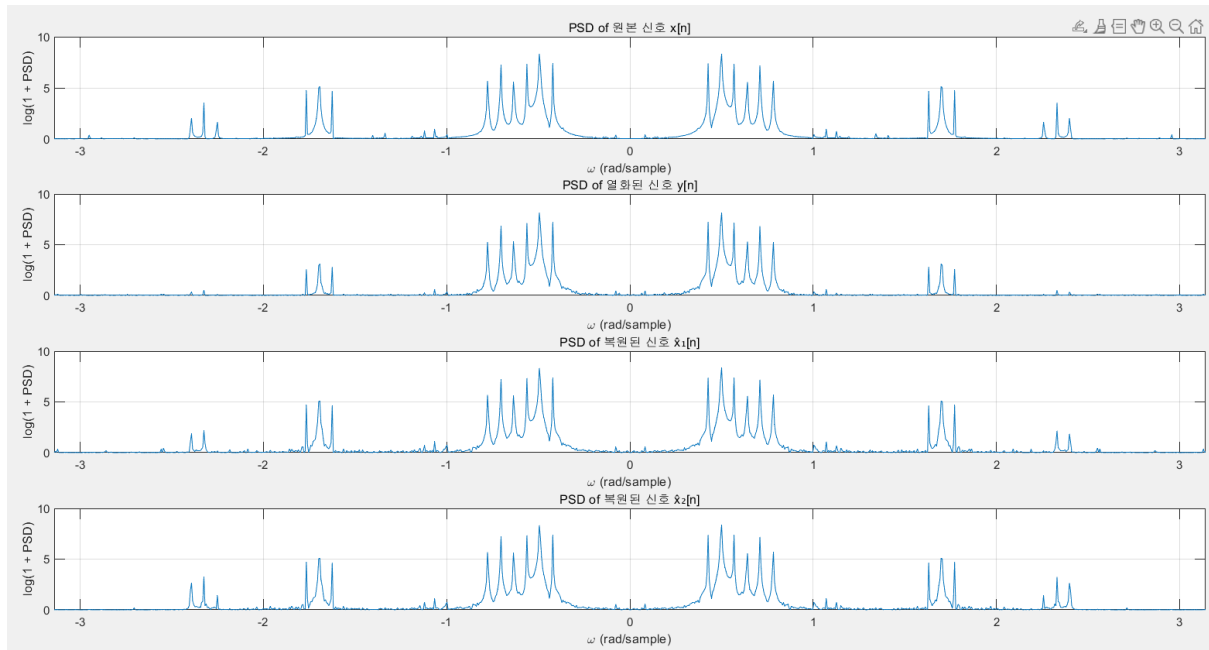








$Y(w) = H(w) * X(w) + U(w)$  로  $y[n]$ 을 구하였을 때, PSD



$\hat{x}_1[n]$ 의 PSD가 좀더 제대로 복원되지 않는 모습을 보인다.

이는 주어진  $h[n]$ 을 FFT를 하며 생기는 DFT 알고리즘 상의 에러라고 생각된다.

따라서, 이번 실습은  $y[n] = h[n] * x[n] + u[n]$ 의 경우 convolution함수를 사용하여 진행하였다.

Winer Filter 적용은 frequency domain에서 하였다.

## Appendix

```
clear;      % 모든 변수 지우기
clc;        % 명령창 지우기
close all;  % 모든 figure 창 닫기

%% Problem 1

% 1. input signal
[x, Fs] = audioread('input.wav'); % Fs = 8000 Hz

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% Playing Sound %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
disp("playing x[n]: input.wav")
sound(x, Fs);
pause(1);
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

% make Time domain
t = (0:length(x)-1) / Fs;

% 2. compute PSD
X_shift = fftshift(fft(x)); % FFT shift
N = length(x);
omega = linspace(-pi, pi, N); % omega domain (-pi ~ pi)
PSD_X_shift = abs(X_shift).^2; % compute power spectrum
PSD_log = log(1 + PSD_X_shift); % log scale
% PSD_log = log10(1 + PSD_X_shift); % log scale

% 3. plot x in time domain
figure;
subplot(2,1,1);
plot(t, x);
xlabel('Time [s]');
ylabel('Amplitude');
title('입력 신호 x[n]');
grid on

% 4. plot PSD (log scale)
subplot(2,1,2);
plot(omega, PSD_log);
xlabel('\omega (rad/sample)');
ylabel('log(1 + PSD)');
title('입력 신호 x[n]의 PSD (log scale)');
xlim([-pi, pi]);
grid on;

%% Problem 2

% 1. filter of system
h = readmatrix('h.txt');

% 2. Compute Frequency Response H(ω)
N = length(h); % FFT length, 보기 좋게 1024로 바뀌도 됨
H_shift = fftshift(fft(h, N));
w = linspace(-pi, pi, N); % ω-domain

% 3. plot h in n domain
figure;
subplot(2,1,1);
stem(0:length(h)-1, h);
xlabel('n');
```

```

ylabel('h[n]');
title('filter h[n] in n domain');
grid on;

% 4. Magnitude plot (linear scale)
subplot(2,1,2);
plot(w, abs(H_shift));
xlabel('\omega (rad/sample)');
ylabel('|H(\omega)|');
title('Frequency Response Magnitude of h[n]');
xlim([-pi pi]);
grid on;

%% Problem 3

% 1. noise
[u, fs_noise] = audioread('noise.wav'); % u[n] :

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% NOISE FFT PLOT %%%%%%%%%%
% % 푸리에 변환 (FFT)
% N = length(u);
% U = fftshift(fft(u)); % 푸리에 변환 + DC 성분 중앙 정렬
% omega = linspace(-pi, pi, N); % 디지털 주파수 축 [-pi, pi]
%
% % 스펙트럼 시각화
% figure;
% plot(omega, abs(U));
% xlabel('\omega (rad/sample)');
% ylabel('|U(\omega)|');
% title('Fourier Transform of Noise Signal (Digital Frequency)');
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% Playing Sound %%%%%%%%%%
disp("playing noise[n]: noise.wav")
sound(u, Fs);
pause(1);
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

% 2-1. If we use convolution for computing y
y = conv(x,h,'same') + u;

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% Playing Sound %%%%%%%%%%
disp("playing y[n]: Degraded input.wav")
sound(y, Fs);
pause(1);
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 2-2. If we use convolution property for computing y
% need to make Length same for convolution property (Zero-padding)
% N_conv = length(x) + length(h) - 1;
% X = fft(x, N_conv);
% H = fft(h, N_conv);
% U = fft(u, N_conv);
%
% convolution in frequency domain -> make Y(w) & y[n]
% Y_freq = X .* H + U; % Y(\omega) = X(\omega) \cdot H(\omega) + U(\omega)
% y = real(ifft(Y_freq)); % 시간 영역 신호로 변환
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

% 3. time domain
t = (0:length(y)-1) / Fs;

% 4. PSD (log scale)
Y_shift = fftshift(fft(y));

```

[illegible]

```

plot(t, x_hat1);
xlabel('Time [s]');
ylabel('Amplitude');
title('복원된 신호 x1(w) (Wiener 필터)');

subplot(2,1,2);
plot(w, PSD_X_hat1_shift_log);
xlabel('\omega (rad/sample)');
ylabel('log(1 + PSD)');
title('복원된 신호 x1[n]의 PSD (log scale)');
xlim([-pi pi]);
grid on;

%% Problem 5 - 원본, 열화, 복원된 신호의 파형 및 PSD 비교

% time domain & w domain should be same to compare
len_min = min([length(x), length(y), length(x_hat1)]);
t_comp = (0:len_min-1) / Fs;
w_comp = linspace(-pi, pi, len_min); % 공통 주파수 축

% fft of x, y, x1
X_comp = fftshift(fft(x(1:len_min)));
Y_comp = fftshift(fft(y(1:len_min)));
Xhat1_comp = fftshift(fft(x_hat1(1:len_min)));

% Compute PSD
PSD_x = abs(X_comp).^2;
PSD_y = abs(Y_comp).^2;
PSD_xhat1 = abs(Xhat1_comp).^2;

% Compute log scale
PSD_x_log = log(1 + PSD_x);
PSD_y_log = log(1 + PSD_y);
PSD_xhat1_log = log(1 + PSD_xhat1);
% PSD_x_log = log10(1 + PSD_x);
% PSD_y_log = log10(1 + PSD_y);
% PSD_xhat1_log = log10(1 + PSD_xhat1);

% === 1. plot x, y, x1 ===
figure;
subplot(3,1,1);
plot(t_comp, x(1:len_min));
xlabel('Time [s]');
ylabel('Amplitude');
title('원본 신호 x[n]');
grid on;

subplot(3,1,2);
plot(t_comp, y(1:len_min));
xlabel('Time [s]');
ylabel('Amplitude');
title('열화된 신호 y[n]');
grid on;

subplot(3,1,3);
plot(t_comp, x_hat1(1:len_min));
xlabel('Time [s]');
ylabel('Amplitude');
title('복원된 신호 x1[n] (Wiener)');
grid on;

% === 2. PSD of x, y, x1 ===
figure;
subplot(3,1,1);
plot(w_comp, PSD_x_log);

```

```

xlabel('\omega (rad/sample)');
ylabel('log(1 + PSD)');
title('PSD of 원본 신호 x[n]');
xlim([-pi pi]);
grid on;

subplot(3,1,2);
plot(w_comp, PSD_y_log);
xlabel('\omega (rad/sample)');
ylabel('log(1 + PSD)');
title('PSD of 열화된 신호 y[n]');
xlim([-pi pi]);
grid on;

subplot(3,1,3);
plot(w_comp, PSD_xhat1_log);
xlabel('\omega (rad/sample)');
ylabel('log(1 + PSD)');
title('PSD of 복원된 신호 x1[n]');
xlim([-pi pi]);
grid on;

%% Problem5 - detail comparison

% time domain & w domain should be same to compare
len_min = min([length(x), length(y), length(x_hat1)]);
t_comp = (0:len_min-1) / Fs;
w_comp = linspace(-pi, pi, len_min);

% cut signals for comparison
x_cut = x(1:len_min);
y_cut = y(1:len_min);
xhat1_cut = x_hat1(1:len_min);

% Compute PSD
X_cut = fftshift(fft(x_cut));
Y_cut = fftshift(fft(y_cut));
Xhat1_cut = fftshift(fft(xhat1_cut));

PSD_x = abs(X_cut).^2;
PSD_y = abs(Y_cut).^2;
PSD_xhat1 = abs(Xhat1_cut).^2;

PSD_x_log = log(1 + PSD_x);
PSD_y_log = log(1 + PSD_y);
PSD_xhat1_log = log(1 + PSD_xhat1);
% PSD_x_log = log10(1 + PSD_x);
% PSD_y_log = log10(1 + PSD_y);
% PSD_xhat1_log = log10(1 + PSD_xhat1);

% === 1. plot in time domain ===
figure;
plot(t_comp, x_cut, 'b-');
hold on;
plot(t_comp, y_cut, 'r--');
plot(t_comp, xhat1_cut, 'k:', 'LineWidth', 1.2);
xlabel('Time [s]');
ylabel('Amplitude');
title('x[n], y[n], x1[n] Time Domain 비교');
legend('원본 x[n]', '열화 y[n]', '복원 x1[n]');
grid on;

% === 2. PSD (log scale) - plot in freq domain ===
figure;
plot(w_comp, PSD_x_log, 'b-');
hold on;
plot(w_comp, PSD_y_log, 'r--');

```

```

plot(w_comp, PSD_xhat1_log, 'k:', 'LineWidth', 1.1);
xlabel('\omega (rad/sample)');
ylabel('log(1 + PSD)');
title('x[n], y[n], x_1[n] PSD 비교 (log scale)');
legend('원본 x[n]', '열화 y[n]', '복원 x_1[n]');
xlim([-pi pi]);
grid on;

%% Problem 6 - Wiener 필터 (Use: S_xx instead of S_yy)

% 1. ReCompute Signals need for Wiener Filter -> need to make length same
X_shift = fftshift(fft(x, N_conv)); % 원본 신호 x[n]의 FFT
Y_shift = fftshift(fft(y, N_conv)); % 열화된 신호 y[n]의 FFT
H_shift = fftshift(fft(h, N_conv)); % 필터
H_conj_shift = conj(H_shift); % H*(\omega)

% 2. Compute PSD
S_xx = abs(X_shift).^2; % S_xx(\omega)
S_uu = 5e-3; % 문제에서 주어진 S_uu

% 3. Compute Wiener Filter (eq.2)
H_wiener2 = (H_conj_shift) ./ (abs(H_shift).^2 + (S_uu ./ S_xx));

% 4. X_hat2(\omega) = H_wiener2 \cdot Y(\omega)
X_hat2_shift = H_wiener2 .* Y_shift;

% 5. IFFT로 x_hat2[n] 얻기
x_hat2 = real(fftshift(fft(X_hat2_shift)));

% 6. time domain & omega domain
t = (0:length(x_hat2)-1) / Fs;
w = linspace(-pi, pi, length(x_hat2));

% 7. Compute PSD
PSD_X_hat2 = abs(X_hat2_shift).^2;
PSD_X_hat2_log = log(1 + PSD_X_hat2); % log scale
% PSD_X_hat2_log = log10(1 + PSD_X_hat2); % log scale

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% Playing Sound %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
disp("playing x_hat2[n]: Restored(Sxx) input.wav")
sound(x_hat2, Fs);
pause(1);
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

% 8. plot
figure;
subplot(2,1,1);
plot(t, x_hat2);
xlabel('Time [s]');
ylabel('Amplitude');
title('복원된 신호 x_2[n] (Wiener 필터 - S_{xx} 사용)');
grid on;

subplot(2,1,2);
plot(w, PSD_X_hat2_log);
xlabel('\omega (rad/sample)');
ylabel('log(1 + PSD)');
title('복원된 신호 x_2[n]의 PSD (log scale)');
xlim([-pi pi]);
grid on;

%% Problem 7 - 원본, 열화, 복원(x1, x2) 신호 비교

```



```

% time domain & w domain should be same to compare
len_min = min([length(x), length(y), length(x_hat1), length(x_hat2)]);
t_comp = (0:len_min-1) / Fs;
w_comp = linspace(-pi, pi, len_min);

% cut signals for comparison
x_cut = x(1:len_min);
y_cut = y(1:len_min);
xhat1_cut = x_hat1(1:len_min);
xhat2_cut = x_hat2(1:len_min);

% Compute PSD
X_cut = fftshift(fft(x_cut));
Y_cut = fftshift(fft(y_cut));
Xhat1_cut = fftshift(fft(xhat1_cut));
Xhat2_cut = fftshift(fft(xhat2_cut));

PSD_x = abs(X_cut).^2;
PSD_y = abs(Y_cut).^2;
PSD_xhat1 = abs(Xhat1_cut).^2;
PSD_xhat2 = abs(Xhat2_cut).^2;

% log scale
PSD_x_log = log(1 + PSD_x);
PSD_y_log = log(1 + PSD_y);
PSD_xhat1_log = log(1 + PSD_xhat1);
PSD_xhat2_log = log(1 + PSD_xhat2);
% PSD_x_log = log10(1 + PSD_x);
% PSD_y_log = log10(1 + PSD_y);
% PSD_xhat1_log = log10(1 + PSD_xhat1);
% PSD_xhat2_log = log10(1 + PSD_xhat2);

%%%%%%%%%% 1. plot in time domain %%%%%%%%%%%
figure;
plot(t_comp, x_cut, 'k', 'LineWidth', 1.1);
hold on;
plot(t_comp, y_cut, 'r--', 'LineWidth', 1.1);
plot(t_comp, xhat1_cut, 'b:', 'LineWidth', 1.1);
plot(t_comp, xhat2_cut, 'g-.');
xlabel('Time [s]');
ylabel('Amplitude');
title('x[n], y[n], x^1[n], x^2[n] 시간 파형 비교');
legend('원본 x[n]', '열화 y[n]', '복원 x^1[n]', '복원 x^2[n]');
grid on;

%%%%%%%%%% 2. PSD Comparison (log scale) %%%%%%%%%%%
figure;
plot(w_comp, PSD_x_log, 'k', 'LineWidth', 1.1);
hold on;
plot(w_comp, PSD_y_log, 'r--', 'LineWidth', 1.1);
plot(w_comp, PSD_xhat1_log, 'b:', 'LineWidth', 1.1);
plot(w_comp, PSD_xhat2_log, 'g-.');
xlabel('\omega (rad/sample)');
ylabel('log(1 + PSD)');
title('x[n], y[n], x^1[n], x^2[n] PSD 비교 (log scale)');
legend('원본 x[n]', '열화 y[n]', '복원 x^1[n]', '복원 x^2[n]');
xlim([-pi pi]);
grid on;

%% Problem 7 - 원본, 열화, 복원(x^1, x^2) 신호 비교 하나 씩

% 1. 길이 정렬 (가장 짧은 길이 기준 자르기)
len_min = min([length(x), length(y), length(x_hat1), length(x_hat2)]);
t_comp = (0:len_min-1) / Fs;
w_comp = linspace(-pi, pi, len_min);

```

```

% 2. 신호 자르기
x_cut = x(1:len_min);
y_cut = y(1:len_min);
xhat1_cut = x_hat1(1:len_min);
xhat2_cut = x_hat2(1:len_min);

% 3. FFT 및 PSD 계산
X_cut = fftshift(fft(x_cut));
Y_cut = fftshift(fft(y_cut));
Xhat1_cut = fftshift(fft(xhat1_cut));
Xhat2_cut = fftshift(fft(xhat2_cut));

PSD_x = abs(X_cut).^2;
PSD_y = abs(Y_cut).^2;
PSD_xhat1 = abs(Xhat1_cut).^2;
PSD_xhat2 = abs(Xhat2_cut).^2;

% log scale 변환
PSD_x_log = log(1 + PSD_x);
PSD_y_log = log(1 + PSD_y);
PSD_xhat1_log = log(1 + PSD_xhat1);
PSD_xhat2_log = log(1 + PSD_xhat2);
% PSD_x_log = log10(1 + PSD_x);
% PSD_y_log = log10(1 + PSD_y);
% PSD_xhat1_log = log10(1 + PSD_xhat1);
% PSD_xhat2_log = log10(1 + PSD_xhat2);

%%%%%%%%%% 1. 시간 영역 파형 비교 %%%%%%%%%%%
figure;

% 1. 원본 신호 x[n]
subplot(4, 1, 1);
plot(t_comp, x_cut);
xlabel('Time [s]');
ylabel('Amplitude');
title('원본 신호 x[n]');
grid on;

% 2. 열화된 신호 y[n]
subplot(4, 1, 2);
plot(t_comp, y_cut);
xlabel('Time [s]');
ylabel('Amplitude');
title('열화된 신호 y[n]');
grid on;

% 3. 복원된 신호 x^1[n]
subplot(4, 1, 3);
plot(t_comp, xhat1_cut);
xlabel('Time [s]');
ylabel('Amplitude');
title('복원된 신호 x^1[n]');
grid on;

% 4. 복원된 신호 x^2[n]
subplot(4, 1, 4);
plot(t_comp, xhat2_cut);
xlabel('Time [s]');
ylabel('Amplitude');
title('복원된 신호 x^2[n]');
grid on;

```

```

##### 2. PSD 로그 스케일 비교 (각각 subplot) #####
figure;

subplot(4, 1, 1);
plot(w_comp, PSD_x_log);
xlabel('\omega (rad/sample)');
ylabel('log(1 + PSD)');
title('PSD of 원본 신호 x[n]');
xlim([-pi pi]);
grid on;

subplot(4, 1, 2);
plot(w_comp, PSD_y_log);
xlabel('\omega (rad/sample)');
ylabel('log(1 + PSD)');
title('PSD of 열화된 신호 y[n]');
xlim([-pi pi]);
grid on;

subplot(4, 1, 3);
plot(w_comp, PSD_xhat1_log);
xlabel('\omega (rad/sample)');
ylabel('log(1 + PSD)');
title('PSD of 복원된 신호  $\hat{x}_1[n]$ ');
xlim([-pi pi]);
grid on;

subplot(4, 1, 4);
plot(w_comp, PSD_xhat2_log);
xlabel('\omega (rad/sample)');
ylabel('log(1 + PSD)');
title('PSD of 복원된 신호  $\hat{x}_2[n]$ ');
xlim([-pi pi]);
grid on;

```